

Contents

Activity 1 – Verify the Kubernetes Cluster	2
Activity 2 – Working with Kubernetes Namespaces	8
Activity 3 – Create Your First Pod	13
Activity 4: Understanding the Pod Lifecycle	19
Activity 5: Working with Labels and Selectors	24
Activity 6: Creating Pods Using YAML Manifests.....	29
Activity 7: Working with Multi-Container Pods	34
Activity 8: Creating and Managing Deployments	40
Activity 9: Exposing Applications Using Kubernetes Services.....	46
Activity 10: Managing Persistent Storage Using Persistent Volumes and Persistent Volume Claims.....	51

Activity 1 – Verify the Kubernetes Cluster

Objective

Verify that the Kubernetes cluster is installed, running, and accessible using kubectl. Learn how to retrieve basic cluster information, check node status, and confirm that the Kubernetes control plane is functioning correctly.

Learning Outcomes

After completing this activity, participants will be able to:

- Verify that kubectl is installed.
- Check the Kubernetes client and server versions.
- Confirm connectivity to the Kubernetes cluster.
- View cluster information.
- List available nodes.
- Verify the status of cluster components.

Estimated Time

20–25 minutes

Prerequisites

- A Kubernetes cluster must be available (Minikube, Kind, K3s, MicroK8s, or a managed Kubernetes cluster).
- kubectl must be installed.
- The cluster should be running.

Before deploying applications to Kubernetes, it is important to verify that the cluster is healthy and accessible.

The primary command-line tool used to interact with Kubernetes is **kubectl**. It communicates with the Kubernetes API Server, allowing users to deploy applications, inspect resources, troubleshoot issues, and manage the cluster.

In this activity, you will verify that:

- kubectl is installed.
- The Kubernetes cluster is reachable.
- The control plane is responding.
- Worker nodes are in the **Ready** state.

- Cluster information can be retrieved successfully.

Step-by-Step Instructions

Step 1: Verify kubectl Installation

Check whether kubectl is installed.

```
kubectl version --client
```

Expected output:

Client Version: v1.xx.x

Kustomize is a native Kubernetes configuration management tool that lets you customize Kubernetes YAML manifests without modifying the original files.

It is built into kubectl, so you don't need to install anything separately in most Kubernetes versions.

Step 2: Verify Client and Server Versions

Display both the client and Kubernetes server versions.

```
kubectl version
```

Expected output:

Client Version: v1.xx.x

Server Version: v1.xx.x

If both versions are displayed, kubectl is successfully communicating with the cluster.

Step 3: Check Cluster Information

Display information about the Kubernetes cluster.

```
kubectl cluster-info
```

Expected output:

Kubernetes control plane is running at https://...

CoreDNS is running at https://...

This confirms that the Kubernetes control plane is accessible.

Step 4: List Cluster Nodes

Display all nodes in the cluster.

```
kubectl get nodes
```

Example output:

NAME	STATUS	ROLES	AGE	VERSION
control-plane	Ready	control-plane	5d	v1.xx.x

For a multi-node cluster:

NAME	STATUS	ROLES	AGE
master	Ready	control-plane	15d
worker-1	Ready	<none>	15d
worker-2	Ready	<none>	15d

Verify that every node shows the **Ready** status.

Step 5: View Detailed Node Information

Display additional details about the nodes.

```
kubectl get nodes -o wide
```

Example output:

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
control-plane	Ready	control-plane	5d	v1.xx.x	172.18.0.2

Notice information such as:

- Internal IP
- Operating System
- Container Runtime
- Kubernetes Version

Step 6: Describe a Node

Display detailed information about a node.

```
kubectl describe node <node-name>
```

Example:

```
kubectl describe node control-plane
```

Observe information such as:

- Labels
- Capacity
- Allocatable resources
- Running Pods
- Conditions
- Events

Step 7: View Kubernetes Context

Display the current Kubernetes context.

```
kubectl config current-context
```

Example output:

```
kind-kind
```

or

```
minikube
```

or

```
kubernetes-admin@kubernetes
```

The current context identifies the cluster to which kubectl is connected.

Step 8: View All Contexts

Display all configured Kubernetes contexts.

```
kubectl config get-contexts
```

Example output:

```
CURRENT  NAME
```

```
*      kind-kind
```

```
      minikube
```

The asterisk (*) indicates the active context.

A context is a named configuration in your kubeconfig file that tells kubectl:

- *Which Kubernetes cluster to connect to.*
- *Which user/credentials to use.*
- *Which namespace (optional) is the default*

Step 9: Display Cluster Configuration

View the Kubernetes configuration file.

```
kubectl config view
```

Observe details such as:

- Clusters
- Users
- Contexts
- API server endpoint

Step 10: Verify Cluster Resources

Display all namespaces.

```
kubectl get namespaces
```

Example output:

NAME	STATUS	AGE
default	Active	5d
kube-system	Active	5d
kube-public	Active	5d
kube-node-lease	Active	5d

This confirms that the cluster is functioning and contains the default namespaces.

Namespace	Purpose
default	The default namespace where your applications are deployed if you don't specify another namespace.

kube-system	Contains Kubernetes system components such as CoreDNS, kube-proxy, CSI drivers, metrics server, etc. You generally shouldn't deploy your own applications here.
kube-public	A publicly readable namespace intended for cluster-wide public resources. It is rarely used in most environments.
kube-node-lease	Stores Lease objects used by nodes to periodically report that they are healthy. This allows the control plane to detect node failures efficiently.

Activity 2 – Working with Kubernetes Namespaces

Objective

Learn how Kubernetes namespaces logically separate cluster resources. Create, view, switch between, and delete namespaces while understanding how they help organize workloads.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of namespaces.
- List existing namespaces.
- Create a new namespace.
- Deploy resources within a namespace.
- View resources in different namespaces.
- Delete a namespace and its associated resources.

Estimated Time

20–25 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.

A **namespace** is a logical partition within a Kubernetes cluster that allows you to organize and isolate resources.

Namespaces are commonly used to:

- Separate development, testing, and production environments.
- Organize applications by teams or projects.
- Apply resource quotas and access controls.
- Avoid naming conflicts between resources.

By default, Kubernetes creates several namespaces for internal use, while user applications are typically deployed in the **default** namespace unless another namespace is specified.

Step-by-Step Instructions

Step 1: View Existing Namespaces

Display all namespaces in the cluster.

```
kubectl get namespaces
```

or

```
kubectl get ns
```

Expected output:

NAME	STATUS	AGE
default	Active	10d
kube-system	Active	10d
kube-public	Active	10d
kube-node-lease	Active	10d

Step 2: Understand the Default Namespaces

The namespaces shown have specific purposes:

Namespace	Purpose
default	Used for user-created resources when no namespace is specified.
kube-system	Contains Kubernetes system components such as CoreDNS and kube-proxy.
kube-public	Stores publicly readable cluster information.
kube-node-lease	Stores node heartbeat information.

Step 3: Create a New Namespace

Create a namespace named **training**.

```
kubectl create namespace training
```

Expected output:

```
namespace/training created
```

Step 4: Verify the Namespace

List all namespaces again.

```
kubectrl get namespaces
```

Expected output:

NAME	STATUS	AGE
default	Active	
kube-system	Active	
kube-public	Active	
kube-node-lease	Active	
training	Active	

Step 5: Describe the Namespace

Display detailed information.

```
kubectrl describe namespace training
```

Observe:

- Name
- Labels
- Status
- Resource quotas (if configured)
- Events

Step 6: Create a Pod Inside the Namespace

Create an Nginx Pod in the **training** namespace.

```
kubectrl run nginx-demo \  
--image=nginx \  
--namespace=training
```

Or

Use the image from the OCIR repository

Expected output:

pod/nginx-demo created

Step 7: View Pods in the Training Namespace

kubectl get pods -n training

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
nginx-demo	1/1	Running	0	20s

Step 8: Compare with the Default Namespace

List Pods in the default namespace.

kubectl get pods

Expected output:

No resources found in default namespace.

Notice that the Pod exists only in the **training** namespace.

Step 9: List Resources in All Namespaces

Display Pods from every namespace.

kubectl get pods --all-namespaces

Expected output:

NAMESPACE	NAME	READY	STATUS
training	nginx-demo	1/1	Running
kube-system	coredns...	1/1	Running

Step 10: Switch the Default Namespace (Current Context)

Instead of specifying -n training every time, update the current context.

kubectl config set-context --current --namespace=training

Verify:

```
kubectl config view --minify | grep namespace
```

Expected output:

```
namespace: training
```

Now simply run:

```
kubectl get pods
```

The Pods displayed will belong to the **training** namespace.

Step 11: Restore the Default Namespace

Reset the current namespace.

```
kubectl config set-context --current --namespace=default
```

Verify:

```
kubectl config view --minify | grep namespace
```

Expected output:

```
namespace: default
```

Step 12: Delete the Namespace

Delete the training namespace.

```
kubectl delete namespace training
```

Expected output:

```
namespace "training" deleted
```

Deleting a namespace automatically deletes all resources within it.

Step 13: Verify Deletion

```
kubectl get namespaces
```

The **training** namespace should no longer appear.

Activity 3 – Create Your First Pod

Objective

Learn how to create, inspect, and manage a Kubernetes Pod using kubectl. Understand the basic structure of a Pod and observe its lifecycle from creation to deletion.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a Pod using kubectl.
- View Pod status.
- Describe a Pod.
- Inspect Pod details.
- Delete a Pod.
- Understand the Pod lifecycle.

Estimated Time

20–25 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.
- The default namespace should be active.

A **Pod** is the smallest deployable unit in Kubernetes. It represents one or more containers that share:

- Network
- Storage
- IP Address
- Hostname

Most Pods contain a single container, although multiple containers can run together inside the same Pod when they need to work closely.

Unlike containers in Podman or Docker, Pods are managed by Kubernetes. Kubernetes monitors Pods and can recreate them if they fail (when managed by higher-level controllers such as Deployments).

Step 1: Verify Existing Pods

Display the Pods in the current namespace.

```
kubectl get pods
```

Expected output:

No resources found in default namespace.

Step 2: Create Your First Pod

Create an Nginx Pod.

```
kubectl run nginx-pod \
```

```
--image=nginx
```

Expected output:

pod/nginx-pod created

Step 3: View the Pod

List all Pods.

```
kubectl get pods
```

Initially, you may observe:

NAME	READY	STATUS	RESTARTS
nginx-pod	0/1	ContainerCreating	0

After a few seconds:

NAME	READY	STATUS	RESTARTS
nginx-pod	1/1	Running	0

Observe the **STATUS** column as the Pod transitions from **ContainerCreating** to **Running**.

Step 4: View Additional Pod Information

Display detailed information.

```
kubectl get pods -o wide
```

Example output:

NAME	READY	STATUS	IP	NODE
nginx-pod	1/1	Running	10.244.0.12	worker-1

Notice:

- Pod IP
- Node Name
- Status
- Ready Containers

Step 5: Describe the Pod

Display detailed information about the Pod.

```
kubectl describe pod nginx-pod
```

Observe the following sections:

- Labels
- Node
- IP Address
- Container Image
- Port
- Conditions
- Events

The **Events** section is especially useful for troubleshooting Pod issues.

Step 6: View the Pod in YAML Format

Display the complete Pod definition.

```
kubectl get pod nginx-pod -o yaml
```

Observe:

- Metadata
- Spec
- Status

- Container configuration

This YAML represents the complete configuration of the Pod.

Step 7: View the Pod in JSON Format

```
kubectl get pod nginx-pod -o json
```

This format is useful for automation and scripting.

Step 8: View Pod Labels

Display Pod labels.

```
kubectl get pod nginx-pod --show-labels
```

Example output:

NAME	READY	STATUS	LABELS
nginx-pod	1/1	Running	run=nginx-pod

Step 9: Watch the Pod

Monitor the Pod continuously.

```
kubectl get pods --watch
```

or

```
kubectl get pods -w
```

Press **Ctrl + C** to stop watching.

Step 10: Delete the Pod

Delete the Pod.

```
kubectl delete pod nginx-pod
```

Expected output:

```
pod "nginx-pod" deleted
```

Step 11: Verify Deletion

List the Pods again.

```
kubect! get pods
```

Expected output:

No resources found in default namespace.

The Pod has been permanently removed.

Troubleshooting

Issue: ImagePullBackOff

The image could not be downloaded.

Verify the image name:

```
kubect! describe pod nginx-pod
```

*Review the **Events** section for image pull errors.*

Issue: Pod remains in Pending state

Check the Pod details:

```
kubect! describe pod nginx-pod
```

Possible causes include:

- *Insufficient cluster resources.*
- *Node scheduling issues.*
- *Missing image.*

Issue: Pod not found

Verify the namespace:

```
kubect! get pods --all-namespaces
```

Ensure you are working in the correct namespace.

Issue: Pod terminates immediately

View the Pod events:

```
kubect! describe pod nginx-pod
```

If the container exits unexpectedly, inspect the logs:

kubect! logs nginx-pod

Activity 4: Understanding the Pod Lifecycle

Objective

Learn about the different phases of a Kubernetes Pod lifecycle and observe how a Pod transitions through various states from creation to termination.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the different Pod phases.
- Observe Pod state transitions.
- Monitor Pod status changes.
- View Pod events.
- Understand why Pods enter different states.
- Delete a Pod and observe its termination.

Estimated Time

20–25 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.

Every Pod in Kubernetes goes through a lifecycle. When a Pod is created, Kubernetes schedules it onto a node, starts its containers, and continuously monitors its health.

A Pod can exist in one of the following phases:

Phase	Description
Pending	The Pod has been accepted by Kubernetes but is waiting to be scheduled or for its container image to be downloaded.
Running	The Pod has been successfully scheduled, and all containers are running.
Succeeded	All containers completed successfully and exited with status code 0.
Failed	One or more containers terminated with an error.
Unknown	Kubernetes cannot determine the Pod's current state, usually due to communication issues with the node.

In addition to these phases, you may encounter container statuses such as:

- ContainerCreating
- CrashLoopBackOff
- ImagePullBackOff
- ErrImagePull
- Terminating

These are not Pod phases but provide additional information about the container's condition.

Step-by-Step Instructions

Step 1: Create a Long-running Pod

Create an Nginx Pod.

```
kubectl run lifecycle-demo \
--image=nginx
```

Verify the Pod.

```
kubectl get pods
```

Initially, the output may look like:

NAME	READY	STATUS	RESTARTS
lifecycle-demo	0/1	ContainerCreating	0

After a few seconds:

NAME	READY	STATUS	RESTARTS
lifecycle-demo	1/1	Running	0

Observe how the Pod moves from **ContainerCreating** to **Running**.

Step 2: Watch the Pod Lifecycle

Monitor the Pod continuously.

```
kubectl get pods --watch
```

or

```
kubectl get pods -w
```

Leave this terminal running.

Open another terminal for the remaining steps.

Press **Ctrl + C** when you finish watching.

Step 3: Describe the Pod

View detailed Pod information.

```
kubect! describe pod lifecycle-demo
```

Observe:

- Conditions
- Events
- Container State
- Pod IP
- Start Time

The **Events** section shows the sequence of actions Kubernetes performed while creating the Pod.

Step 4: View the Pod Logs

Display the logs.

```
kubect! logs lifecycle-demo
```

For an Nginx Pod, there may be little or no output because no requests have been made yet.

Step 5: Observe the Running Phase

Verify the Pod is running.

```
kubect! get pod lifecycle-demo
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
lifecycle-demo	1/1	Running	0	2m

Step 6: Create a Pod That Completes Successfully

Run a Pod that prints a message and exits.

```
kubectrl run completed-pod \
```

```
--image=busybox \
```

```
--restart=Never \
```

```
-- echo "Hello Kubernetes"
```

Check the Pod status.

```
kubectrl get pods
```

Expected output after a few moments:

NAME	READY	STATUS	RESTARTS
------	-------	--------	----------

completed-pod	0/1	Completed	0
---------------	-----	-----------	---

The Pod has successfully completed its task.

Step 7: Create a Pod That Fails

Create a Pod with an invalid command.

```
kubectrl run failed-pod \
```

```
--image=busybox \
```

```
--restart=Never \
```

```
-- invalidcommand
```

Verify the status.

```
kubectrl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS
------	-------	--------	----------

failed-pod	0/1	Error	0
------------	-----	-------	---

Describe the Pod.

```
kubectrl describe pod failed-pod
```

View the logs.

```
kubectrl logs failed-pod
```

Observe the error message indicating that the command could not be found.

Step 8: Observe the Terminating State

Delete the running Pod.

```
kubect! delete pod lifecycle-demo
```

If you are still running:

```
kubect! get pods --watch
```

you may briefly observe:

STATUS

Terminating

before the Pod disappears.

Step 9: Verify Pod Deletion

Display all Pods.

```
kubect! get pods
```

Only the completed and failed Pods should remain.

Step 10: Delete the Remaining Pods

Delete both Pods.

```
kubect! delete pod completed-pod failed-pod
```

Verify.

```
kubect! get pods
```

Expected output:

No resources found in default namespace.

Activity 5: Working with Labels and Selectors

Objective

Learn how to use labels to organize Kubernetes resources and selectors to filter and manage those resources efficiently.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of labels and selectors.
- Create Pods with labels.
- View labels assigned to resources.
- Add, modify, and remove labels.
- Filter resources using label selectors.
- Use multiple label selectors.

Estimated Time

20–25 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.

Labels are **key-value pairs** attached to Kubernetes objects such as Pods, Deployments, Services, and Nodes.

Labels help organize and categorize resources without affecting how the application runs.

Common labels include:

- Application name
- Environment
- Version
- Team
- Department

Example:

app=nginx

environment=development

version=v1

team=devops

Selectors are used to identify resources based on their labels.

For example:

- A Service uses selectors to identify which Pods should receive traffic.
- A Deployment uses selectors to manage its Pods.

Step 1: Verify Existing Pods

```
kubectl get pods
```

Expected output:

No resources found in default namespace.

Step 2: Create the First Pod with Labels

```
kubectl run web-app \
```

```
--image=nginx \
```

```
--labels=app=web,environment=development,version=v1
```

Verify:

```
kubectl get pods
```

Step 3: Create the Second Pod

```
kubectl run api-app \
```

```
--image=nginx \
```

```
--labels=app=api,environment=development,version=v1
```

Step 4: Create the Third Pod

```
kubectl run web-production \
```

```
--image=nginx \
```

```
--labels=app=web,environment=production,version=v2
```

Step 5: View Pod Labels

Display all Pods along with their labels.

```
kubectl get pods --show-labels
```

Example output:

NAME	READY	STATUS	LABELS
web-app	1/1	Running	app=web,environment=development,version=v1
api-app	1/1	Running	app=api,environment=development,version=v1
web-production	1/1	Running	app=web,environment=production,version=v2

Step 6: Describe a Pod

View detailed information about one Pod.

```
kubectl describe pod web-app
```

Locate the **Labels** section.

Step 7: Filter Pods by Label

Display only Pods where:

```
app=web
```

```
kubectl get pods -l app=web
```

Expected output:

NAME
web-app
web-production

Step 8: Filter Using Another Label

Display only development Pods.

```
kubectl get pods -l environment=development
```

Expected output:

NAME

web-app

api-app

Step 9: Use Multiple Label Selectors

Display Pods matching both labels.

```
kubectl get pods -l app=web,environment=development
```

Expected output:

NAME

web-app

Step 10: Add a New Label

Add a team label.

```
kubectl label pod web-app team=devops
```

Verify:

```
kubectl get pod web-app --show-labels
```

Step 11: Update an Existing Label

Modify the version label.

```
kubectl label pod web-app version=v2 --overwrite
```

Verify:

```
kubectl get pod web-app --show-labels
```

Step 12: Remove a Label

Remove the team label.

```
kubectl label pod web-app team-
```

Verify:

```
kubectl get pod web-app --show-labels
```

Notice that the team label is no longer present.

Step 13: Display Labels in Custom Columns

Display only the Pod name and labels.

```
kubectl get pods --show-labels
```

Alternatively:

```
kubectl get pods -L environment,version
```

Example output:

NAME	STATUS	ENVIRONMENT	VERSION
web-app	Running	development	v2
api-app	Running	development	v1
web-production	Running	production	v2

Step 14: Delete All Pods

```
kubectl delete pod web-app api-app web-production
```

Verify:

```
kubectl get pods
```

Expected output:

No resources found in default namespace.

Activity 6: Creating Pods Using YAML Manifests

Learn how to create, inspect, update, and delete Kubernetes Pods using YAML manifest files instead of imperative commands.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the structure of a Kubernetes YAML manifest.
- Create a Pod using a YAML file.
- Apply a manifest using `kubectl apply`.
- View and inspect the created Pod.
- Modify the YAML file and reapply changes.
- Delete resources using the manifest file.

Estimated Time

25–30 minutes

Prerequisites

- A running Kubernetes cluster.
- `kubectl` configured and connected to the cluster.
- Basic knowledge of Pods.

While `kubectl run` is useful for quickly creating Pods, Kubernetes resources are typically managed using **YAML manifest files**.

A YAML manifest describes the desired state of a resource. Kubernetes continuously works to ensure that the actual state matches the desired state defined in the manifest.

Using YAML manifests offers several advantages:

- Easy to version control using Git.
- Reusable across environments.
- Easy to modify and update.
- Infrastructure can be managed as code.
- Simplifies collaboration among team members.

Step-by-Step Instructions

Step 1: Create a Working Directory

Create a directory for this activity.

```
mkdir k8s-yaml
```

```
cd k8s-yaml
```

Verify the current directory.

```
pwd
```

Step 2: Create the YAML File

Create a file named nginx-pod.yaml.

```
nano nginx-pod.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: nginx-pod
```

```
  labels:
```

```
    app: nginx
```

```
    environment: development
```

```
spec:
```

```
  containers:
```

```
  - name: nginx-container
```

```
    image: nginx:latest
```

```
    ports:
```

```
    - containerPort: 80
```

Save the file.

Step 3: Explain the YAML File

Field	Description
apiVersion	Specifies the Kubernetes API version used for the resource.
kind	Defines the type of resource to create (Pod).
metadata	Contains information such as the resource name and labels.
spec	Defines the desired state of the Pod.
containers	Lists the containers that will run inside the Pod.
name	Name of the container.
image	Container image to use.
containerPort	Port exposed by the container.

Step 4: Create the Pod

Apply the manifest.

```
kubectl apply -f nginx-pod.yaml
```

Expected output:

```
pod/nginx-pod created
```

Step 5: Verify the Pod

```
kubectl get pods
```

Example output:

```
NAME      READY  STATUS   RESTARTS  AGE
nginx-pod  1/1    Running  0          15s
```

Step 6: View Detailed Information

```
kubectl describe pod nginx-pod
```

Observe:

- Labels
- Image
- IP Address

- Node
- Events

Step 7: View the YAML Generated by Kubernetes

`kubectl get pod nginx-pod -o yaml`

Compare the output with your original YAML file.

Notice that Kubernetes automatically adds fields such as:

- UID
- Resource Version
- Status
- Creation Timestamp

Step 8: Modify the Manifest

Edit the YAML file.

`nano nginx-pod.yaml`

Add another label.

labels:

app: nginx

environment: development

team: devops

Save the file.

Step 9: Apply the Updated Manifest

`kubectl apply -f nginx-pod.yaml`

Expected output:

pod/nginx-pod configured

The Pod metadata is updated with the new label.

Step 10: Verify the Updated Label

```
kubectl get pod nginx-pod --show-labels
```

Expected output:

NAME	READY	STATUS	LABELS
------	-------	--------	--------

nginx-pod	1/1	Running	app=nginx,environment=development,team=devops
-----------	-----	---------	---

Step 11: View the Manifest Without Creating Resources

Validate the manifest.

```
kubectl apply --dry-run=client -f nginx-pod.yaml
```

Expected output:

```
pod/nginx-pod created (dry run)
```

This verifies the manifest without creating or modifying any resources.

Step 12: Delete the Pod Using the Manifest

```
kubectl delete -f nginx-pod.yaml
```

Expected output:

```
pod "nginx-pod" deleted
```

Step 13: Verify Deletion

```
kubectl get pods
```

Expected output:

No resources found in default namespace.

Activity 7: Working with Multi-Container Pods

Objective

Learn how multiple containers can run inside a single Kubernetes Pod and understand how they share networking and storage resources.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the concept of a multi-container Pod.
- Create a Pod containing multiple containers.
- Verify all containers are running.
- Execute commands inside specific containers.
- View logs from individual containers.
- Understand how containers within the same Pod communicate.

Estimated Time

25–30 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.

Although many Pods contain only a single container, Kubernetes allows multiple containers to run within the same Pod.

Containers inside the same Pod:

- Share the same network namespace.
- Share the same IP address.
- Can communicate using localhost.
- Can share storage through Kubernetes volumes.
- Are always scheduled together on the same node.

Multi-container Pods are commonly used for:

- Sidecar containers
- Logging agents

- Monitoring agents
- Proxy containers
- Configuration synchronization

In this activity, you will create a Pod with two containers:

- Nginx Web Server
- BusyBox Utility Container

Both containers will share a common volume.

Step-by-Step Instructions

Step 1: Create a Working Directory

```
mkdir multi-container
```

```
cd multi-container
```

Step 2: Create the YAML Manifest

Create a file named multi-container-pod.yaml.

```
nano multi-container-pod.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: multi-container-demo
```

```
spec:
```

```
  volumes:
```

```
  - name: shared-data
```

```
    emptyDir: {}
```

```
  containers:
```

```
- name: nginx
  image: nginx:latest
  volumeMounts:
    - name: shared-data
      mountPath: /usr/share/nginx/html
```

```
- name: busybox
  image: busybox:latest
  command:
    - sh
    - -c
    - while true; do echo "Hello from BusyBox" > /shared/index.html; sleep 5; done
  volumeMounts:
    - name: shared-data
      mountPath: /shared
```

Save the file.

Step 3: Create the Pod

```
kubectl apply -f multi-container-pod.yaml
```

Expected output:

```
pod/multi-container-demo created
```

Step 4: Verify the Pod

```
kubectl get pods
```

Example output:

NAME	READY	STATUS
multi-container-demo	2/2	Running

Notice the **READY** column shows **2/2**, indicating that both containers are running.

Step 5: View Detailed Pod Information

```
kubectl describe pod multi-container-demo
```

Observe that two containers are listed:

- nginx
- busybox

Also observe:

- Shared volume
- Container status
- Events

Step 6: View Container Names

```
kubectl get pod multi-container-demo -o jsonpath='{.spec.containers[*].name}'
```

Expected output:

```
nginx busybox
```

Step 7: Execute a Command Inside the Nginx Container

```
kubectl exec -it multi-container-demo -c nginx -- sh
```

View the shared directory.

```
ls /usr/share/nginx/html
```

Display the webpage.

```
cat /usr/share/nginx/html/index.html
```

Expected output:

```
Hello from BusyBox
```

Exit the container.

```
exit
```

Step 8: Execute a Command Inside the BusyBox Container

```
kubectl exec -it multi-container-demo -c busybox -- sh
```

View the shared file.

```
cat /shared/index.html
```

Exit.

```
exit
```

Step 9: View Logs from the BusyBox Container

```
kubectl logs multi-container-demo -c busybox
```

Since BusyBox continuously updates the shared file, logs may be minimal depending on the container's command.

Step 10: View Logs from the Nginx Container

```
kubectl logs multi-container-demo -c nginx
```

Each container maintains its own logs.

Step 11: Verify Shared Networking

Open a shell inside the BusyBox container.

```
kubectl exec -it multi-container-demo -c busybox -- sh
```

Install wget is not possible in BusyBox, so simply verify that Nginx is listening.

```
wget -qO- http://localhost
```

Expected output:

```
Hello from BusyBox
```

This demonstrates that both containers communicate using **localhost** because they share the same network namespace.

Exit.

```
exit
```

Step 12: Delete the Pod

```
kubectl delete -f multi-container-pod.yaml
```

Step 13: Verify Deletion

```
kubectl get pods
```

Expected output:

No resources found in default namespace.

Activity 8: Creating and Managing Deployments

Objective

Learn how to create, inspect, update, scale, and delete Kubernetes Deployments. Understand how Deployments manage Pods and provide self-healing and rolling update capabilities.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a Deployment using a YAML manifest.
- View Deployment details.
- Understand the relationship between Deployments, ReplicaSets, and Pods.
- Scale a Deployment.
- Update the container image.
- Perform a rolling update.
- Delete a Deployment.

Estimated Time

30–35 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.

A **Deployment** is one of the most commonly used Kubernetes resources.

Unlike standalone Pods, a Deployment automatically manages Pods by:

- Maintaining the desired number of replicas.
- Recreating failed Pods.
- Performing rolling updates.
- Supporting rollbacks.
- Scaling applications.

Deployment hierarchy:

Deployment

|



ReplicaSet

|



Pods

Users interact with the **Deployment**, while Kubernetes automatically creates and manages the ReplicaSet and Pods.

Step-by-Step Instructions

Step 1: Create a Working Directory

```
mkdir deployment-demo
```

```
cd deployment-demo
```

Step 2: Create the Deployment YAML

Create a file named nginx-deployment.yaml.

```
nano nginx-deployment.yaml
```

Add the following content.

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
app: nginx
```

```
template:
```

```
  metadata:
```

```
    labels:
```

```
      app: nginx
```

```
  spec:
```

```
    containers:
```

```
      - name: nginx
```

```
        image: nginx:latest
```

```
    ports:
```

```
      - containerPort: 80
```

Save the file.

Step 3: Create the Deployment

```
kubectl apply -f nginx-deployment.yaml
```

Expected output:

```
deployment.apps/nginx-deployment created
```

Step 4: View the Deployment

```
kubectl get deployments
```

Example output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	3/3	3	3	20s

Step 5: View ReplicaSets

```
kubectl get replicaset
```

Example output:

NAME	DESIRED	CURRENT	READY
nginx-deployment-6b8d45b4c	3	3	3

Notice that Kubernetes automatically created a ReplicaSet.

Step 6: View Pods

```
kubectl get pods
```

Example output:

NAME	READY	STATUS
nginx-deployment-6b8d45b4c-4f7mz	1/1	Running
nginx-deployment-6b8d45b4c-k9rtd	1/1	Running
nginx-deployment-6b8d45b4c-x8jlv	1/1	Running

Observe that three Pods were automatically created.

Step 7: Describe the Deployment

```
kubectl describe deployment nginx-deployment
```

Observe:

- Replicas
- Strategy
- Selector
- Pod Template
- Events

Step 8: Scale the Deployment

Increase the number of replicas to five.

```
kubectl scale deployment nginx-deployment --replicas=5
```

Verify.

```
kubectl get deployments
```

View the Pods.

```
kubectl get pods
```

Five Pods should now be running.

Step 9: Scale Down

Reduce the number of replicas.

```
kubectl scale deployment nginx-deployment --replicas=2
```

Verify.

```
kubectl get pods
```

Only two Pods should remain.

Step 10: Update the Deployment Image

Update the Deployment to use a specific Nginx version.

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.27
```

Expected output:

```
deployment.apps/nginx-deployment image updated
```

Step 11: Monitor the Rolling Update

Watch the rollout.

```
kubectl rollout status deployment/nginx-deployment
```

Expected output:

```
deployment "nginx-deployment" successfully rolled out
```

Observe that Pods are updated gradually instead of all at once.

Step 12: View Deployment History

```
kubectl rollout history deployment/nginx-deployment
```

Example output:

deployment.apps/nginx-deployment

REVISION CHANGE-CAUSE

1

2

Step 13: Verify the Running Image

Describe one Pod.

kubectl describe pod <pod-name>

or

kubectl get pods

Choose any Pod and verify that it uses:

nginx:1.27

Step 14: Delete the Deployment

kubectl delete deployment nginx-deployment

Expected output:

deployment.apps "nginx-deployment" deleted

Step 15: Verify Cleanup

kubectl get deployments

kubectl get replicaset

kubectl get pods

All resources created by the Deployment should be removed.

Activity 9: Exposing Applications Using Kubernetes Services

Objective

Learn how to expose Kubernetes applications using Services and understand the differences between ClusterIP, NodePort, and LoadBalancer Services.

Learning Outcomes

After completing this activity, participants will be able to:

- Create a Deployment for an application.
- Expose the Deployment using a Service.
- Understand the purpose of ClusterIP and NodePort Services.
- Access the application through a NodePort.
- Verify Service endpoints.
- Delete the Service and Deployment.

Estimated Time

30–35 minutes

Prerequisites

- A running Kubernetes cluster (Minikube, Kind, Docker Desktop, etc.).
- kubectl configured and connected to the cluster.

Pods are temporary resources with dynamic IP addresses. If a Pod is deleted or recreated, its IP changes.

A **Service** provides a stable network endpoint that allows clients to communicate with one or more Pods without knowing their individual IP addresses.

Common Service types include:

Service Type Purpose

ClusterIP Exposes the application only inside the cluster.

NodePort Exposes the application on a port of every cluster node.

LoadBalancer Exposes the application using an external cloud load balancer.

In this activity, you will expose an Nginx Deployment using a **NodePort Service**.

Step-by-Step Instructions

Step 1: Create a Working Directory

```
mkdir service-demo
```

```
cd service-demo
```

Step 2: Create the Deployment YAML

Create a file named nginx-deployment.yaml.

```
nano nginx-deployment.yaml
```

Add the following content.

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx
```

```
image: nginx:latest
```

```
ports:
```

```
- containerPort: 80
```

Save the file.

Step 3: Create the Deployment

```
kubectl apply -f nginx-deployment.yaml
```

Verify.

```
kubectl get deployments
```

```
kubectl get pods
```

Three Pods should be running.

Step 4: Expose the Deployment

Create a NodePort Service.

```
kubectl expose deployment nginx-deployment \
```

```
--type=NodePort \
```

```
--port=80 \
```

```
--target-port=80
```

Expected output:

```
service/nginx-deployment exposed
```

Step 5: View the Service

```
kubectl get services
```

Example output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx-deployment	NodePort	10.96.85.110	<none>	80:31245/TCP

Notice the automatically assigned **NodePort** (for example, 31245).

Step 6: Describe the Service

```
kubectl describe service nginx-deployment
```

Observe:

- Selector
- Cluster IP
- Port
- Target Port
- NodePort
- Endpoints

Step 7: Verify Service Endpoints

kubectl get endpoints

Example output:

NAME	ENDPOINTS
nginx-deployment	10.244.0.5:80,10.244.0.6:80,10.244.0.7:80

These are the IP addresses of the Pods behind the Service.

Step 8: Access the Application

For **Minikube**:

minikube service nginx-deployment

For **Docker Desktop Kubernetes**:

http://localhost:<NodePort>

For **Kind**:

Use the node IP with the NodePort.

http://<Node-IP>:<NodePort>

You should see the default Nginx welcome page.

Step 9: Scale the Deployment

Increase the number of replicas.

kubectl scale deployment nginx-deployment --replicas=5

Verify.

kubectl get pods

Step 10: Verify Updated Endpoints

kubectl get endpoints

Notice that the Service automatically includes the newly created Pods.

Step 11: Delete One Pod

List the Pods.

```
kubectl get pods
```

Delete one Pod.

```
kubectl delete pod <pod-name>
```

Verify.

```
kubectl get endpoints
```

The Service automatically updates its endpoints after the replacement Pod becomes ready.

Step 12: View the Service in YAML

```
kubectl get service nginx-deployment -o yaml
```

Observe:

- Cluster IP
- Ports
- Selector
- Session Affinity
- Type

Step 13: Delete the Service

```
kubectl delete service nginx-deployment
```

Step 14: Delete the Deployment

```
kubectl delete deployment nginx-deployment
```

Step 15: Verify Cleanup

```
kubectl get all
```

No resources created during this activity should remain.

Activity 10: Managing Persistent Storage Using Persistent Volumes and Persistent Volume Claims

Objective

Learn how to provide persistent storage to Kubernetes applications using Persistent Volumes (PV) and Persistent Volume Claims (PVC), ensuring that application data survives Pod deletion.

Learning Outcomes

After completing this activity, participants will be able to:

- Understand the purpose of Persistent Volumes and Persistent Volume Claims.
- Create a Persistent Volume.
- Create a Persistent Volume Claim.
- Mount persistent storage into a Pod.
- Verify that data persists after the Pod is deleted.
- Delete storage resources safely.

Estimated Time

35–40 minutes

Prerequisites

- A running Kubernetes cluster.
- kubectl configured and connected to the cluster.
- A cluster that supports Persistent Volumes (Minikube, Kind, Docker Desktop, OKE, etc.).

Containers are ephemeral by nature. When a Pod is deleted, any data stored inside the container's writable layer is lost.

Kubernetes provides persistent storage using two resources:

- **Persistent Volume (PV)** – A storage resource available in the cluster.
- **Persistent Volume Claim (PVC)** – A request for storage made by a Pod.

A Pod uses a PVC instead of directly accessing a PV. Kubernetes binds the PVC to a suitable PV automatically.

In production environments such as OKE, Persistent Volumes are typically backed by OCI Block Volumes or OCI File Storage through Storage Classes.

Step-by-Step Instructions

Step 1: Create a Working Directory

```
mkdir storage-demo
```

```
cd storage-demo
```

Step 2: Create a Persistent Volume

Create a file named pv.yaml.

```
nano pv.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: PersistentVolume
```

```
metadata:
```

```
  name: training-pv
```

```
spec:
```

```
  capacity:
```

```
    storage: 1Gi
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  persistentVolumeReclaimPolicy: Retain
```

```
  hostPath:
```

```
    path: /tmp/training-storage
```

Save the file.

Create the PV.

```
kubectl apply -f pv.yaml
```

Verify.

```
kubect! get pv
```

Expected output:

NAME	CAPACITY	ACCESS MODES	STATUS
training-pv	1Gi	RWO	Available

Step 3: Create a Persistent Volume Claim

Create a file named pvc.yaml.

```
nano pvc.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: training-pvc
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
resources:
```

```
  requests:
```

```
    storage: 1Gi
```

Save the file.

Apply the manifest.

```
kubect! apply -f pvc.yaml
```

Verify.

```
kubect! get pvc
```

Example output:

NAME	STATUS	VOLUME
training-pvc	Bound	training-pv

Step 4: Verify the Binding

```
kubectl get pv
```

The PV status should now be **Bound**.

Step 5: Create a Pod that Uses the PVC

Create a file named storage-pod.yaml.

```
nano storage-pod.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: storage-demo
```

```
spec:
```

```
  containers:
```

```
  - name: nginx
```

```
    image: nginx:latest
```

```
  volumeMounts:
```

```
  - name: app-storage
```

```
    mountPath: /usr/share/nginx/html
```

```
  volumes:
```

```
- name: app-storage
persistentVolumeClaim:
  claimName: training-pvc
```

Save the file.

Create the Pod.

```
kubectl apply -f storage-pod.yaml
```

Verify.

```
kubectl get pods
```

Step 6: Create a File Inside the Volume

Open a shell.

```
kubectl exec -it storage-demo -- sh
```

Create a file.

```
echo "Persistent Storage Demo" > /usr/share/nginx/html/index.html
```

Verify.

```
cat /usr/share/nginx/html/index.html
```

Exit.

```
exit
```

Step 7: Delete the Pod

```
kubectl delete pod storage-demo
```

Verify.

```
kubectl get pods
```

The Pod should no longer exist.

Step 8: Recreate the Pod

```
kubectl apply -f storage-pod.yaml
```

Wait until it is running.

```
kubectl get pods
```

Step 9: Verify Persistent Data

Open the shell.

```
kubectl exec -it storage-demo -- sh
```

Display the file.

```
cat /usr/share/nginx/html/index.html
```

Expected output:

Persistent Storage Demo

The file still exists because it is stored on the Persistent Volume.

Exit.

```
exit
```

Step 10: View Storage Resources

```
kubectl get pv
```

```
kubectl get pvc
```

Describe the PVC.

```
kubectl describe pvc training-pvc
```

Observe:

- Capacity
- Access Mode
- Bound Volume
- Events

Step 11: Delete the Pod

```
kubectl delete pod storage-demo
```

Step 12: Delete the PVC


```
kubectrl delete pvc training-pvc
```

Because the PV uses the **Retain** reclaim policy, the Persistent Volume is not deleted automatically.

Verify.

```
kubectrl get pv
```

The PV should remain.

Step 13: Delete the Persistent Volume

```
kubectrl delete pv training-pv
```

Step 14: Verify Cleanup

```
kubectrl get pv
```

```
kubectrl get pvc
```

No storage resources created during this activity should remain.